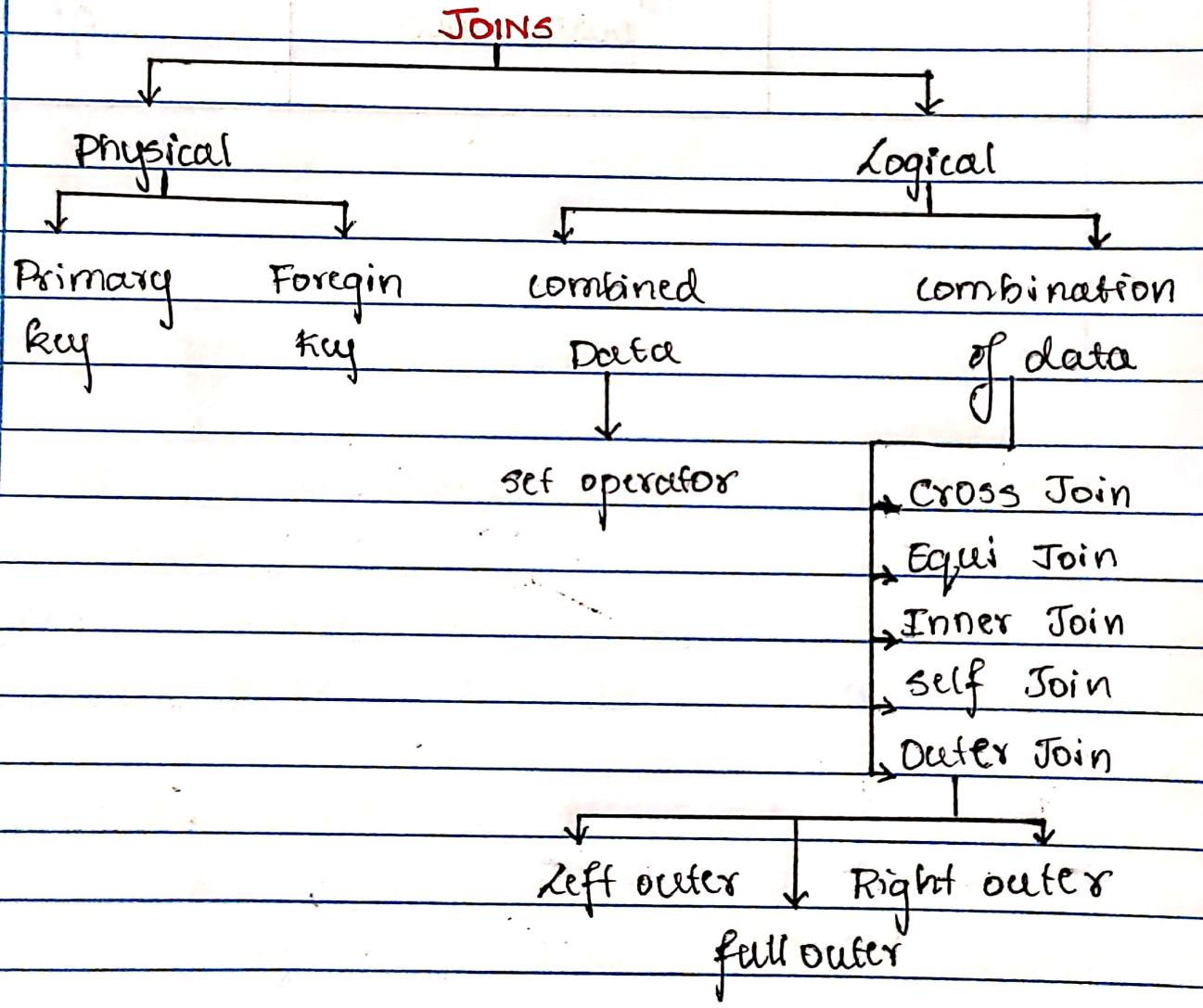
apsara

Date: —·—·——

JOINS

A Join is used to retrieve data from two or more tables based on a relationship between them



Scanned with OKEN Scanner

apsara

Date: __/__/____

Imagin we have these two tables.

Employee table.

emp_id	ename	deptno
101	Arun	10
102	Priya	20
103	Rahul	30

Department table.

deptno	dname	location
10	HR	Chennai
20	IT	Bengaluru
40	Sales	Mumbai

Notice that deptno connects to two tables

- Employee Arun belongs to department 10 (HR)
- Employee Priya belongs to department 20 (IT)
- Rahul belongs to department 30, which is missing from the department table
- Sales, department 40, has no employee in our employee table

A Join helps us combine information from these tables

Page No.: _____

Scanned with OKEN Scanner

apsara

Date: __/__/____

Types of JOINS

1. CROSS JOIN

Combines every row of the first table with every row of the second table, It is also called a cartesian join.

Ex: suppose we have two tables.

1. Employee table

emp_id	ename
101	Arun
102	Priya
103	Rahul

2. Department table

deptno	dname
10	HR
20	IT
30	Sales

Syntax :
```
SELECT columns
FROM table1
CROSS JOIN table2;
```

Ex :
```
SELECT e.name, d.dname
FROM emp e
CROSS JOIN dept d;
```

Page No.: ___

Scanned with OKEN Scanner

Result: There will be 3x3 = 9 combination

ename	dname
Arun	HR
Arun	IT
Arun	sales
Priya	HR
Priya	IT
Priya	Sales
Rahul	HR
Rahul	IT
Rahul	sales.

CROSS JOIN does not require a joining condition

2. EQUI JOIN

EQUI JOIN matches records from two tables using the equality (=) operator

syntax:
```
SELECT columns
FROM table1, table2
WHERE table1.common_column =
  table2.common_column;
```


Scanned with OKEN Scanner

Ex: SELECT e.name , d.dname
FROM emp.e , dept d
WHERE e.deptno = d.deptno ;

Result:

ename	dname
Arun	HR
Priya	IT

why only two rows ? : Because department number 10 and and 20 matches in both tables.
Rahul's department 30 has no matching department record, and sales has no employee.

EQUI JOIN uses = to match records.

3. INNER JOIN

This returns only the rows that satisfy the joining condition in both tables.

Syntax: SELECT columns
FROM table1
INNER JOIN table2

Scanned with OKEN Scanner

ON table1. common_column = table2. common_column;

Ex: SELECT e.emp_id,
e.ename,
d.dname
FROM emp e
INNER JOIN dept d
ON e.deptno = d.deptno;

Result:

emp_id	ename	dname
101	Arun	HR
102	Priya	IT

The result is the same as our EQUI JOIN example because both use the equality condition.

Difference b/w EQUI JOIN and INNER JOIN

- Equi JOIN specifically uses the (=) operator
- INNER JOIN describes the matching row behavior and can use other comparison conditions too, depending on the (ON) condition

ex: an INNER JOIN can use (>) as a condition although that is not an EQUI JOIN.

Scanned with OKEN Scanner

apsara

Date: —/—/—

4. SELF JOIN

This joins a table with itself. It is useful when rows in the same table are related, such as employees and their managers.

Suppose the employee table contains

emp_id	emp_name	manager_id
101	Arun	103
102	Priya	103
103	Rahul	Null

Syntax:
```
SELECT a.column_name,
       b.column_name
FROM table_name a
JOIN table_name b
ON a.common_column =
   b.common_column
```

Ex:
```
SELECT e.emp_name AS Employee,
       m.emp_name AS Manager
FROM emp e
JOIN emp m
ON e.manager_id = m.emp_id;
```


Scanned with OKEN Scanner

Result :

Employee	Manager
Arun	Rahul
Priya	Rahul

aliases :

- e represents the employee's row
- m represents the manager's row
- e.manager_id = m.emp_id matches an employee to their manager

SELF JOIN = same table joined with itself.

5. OUTER JOIN

Outer joins preserve unmatched rows from one or both tables.
Outer JOIN is divided into three types.

a. LEFT OUTER JOIN

Returns all employees from the left table and matching department info from the right table

Ex :
```
SELECT e.ename, d.dname
FROM emp e
LEFT OUTER JOIN dept d
ON e.deptno = d.deptno
```


Scanned with OKEN Scanner

Ex: Employee table

emp_id	ename	deptno
101	Arun	10
102	Priya	20
103	Rahul	30

Department table

deptno	dname
10	HR
20	IT
40	Sales.

Syntax:

```
SELECT columns
FROM table1
LEFT OUTER JOIN table2
ON table1.column_name =
    table2.column_name;
```

o/p:

ename	dname
ARUN	HR
Priya	IT
Rahul	Null

Rahul is included because all rows from the left table (emp) are preserved

Scanned with OKEN Scanner

his department has no match, so dname is NULL

6. Right Outer Join

Returns all departments from the right table and matching employee information from the left table

Syntax :
```
SELECT columns
FROM table1
RIGHT OUTER JOIN table2
ON table1.column_name =
table2.column_name;
```

Ex:
```
SELECT e.ename, d.dname
FROM emp e
RIGHT OUTER JOIN dept d
ON e.deptno = d.deptno;
```

Result :

ename	dname
Areen	HR
Priya	IT
NULL	Sales

Sales is included because all rows from the right table (dept) are preserved

Scanned with OKEN Scanner

No employee matches if so @name is NULL

c. FULL OUTER JOIN

Returns all matching and unmatched rows from both tables

Syntax :
```
SELECT columns
FROM table1
FULL OUTER JOIN table2
ON table1.column_name =
 table2.column_name ;
```

Ex :
```
SELECT e.ename, d.dname
FROM emp e
FULL OUTER JOIN dept d
ON e.deptno = d.deptno;
```

o/p :

ename	dname
Arun	HR
Priya	IT
Rahul	Null
Null	Sales

NOTE : MySQL doesn't directly support FULL OUTER JOIN, you can combine LEFT JOIN

Scanned with OKEN Scanner

and a RIGHT JOIN with UNION :

```
SELECT e.ename, d.dname
FROM emp e
LEFT JOIN dept d
ON e.deptno = d.deptno

UNION

SELECT e.name, d.dname
FROM emp e
RIGHT JOIN dept d
ON e.deptno = d.deptno;
```


Scanned with OKEN Scanner